GADAA DIGITAL

CHAPTER ONE

Introduction to Computer Programming

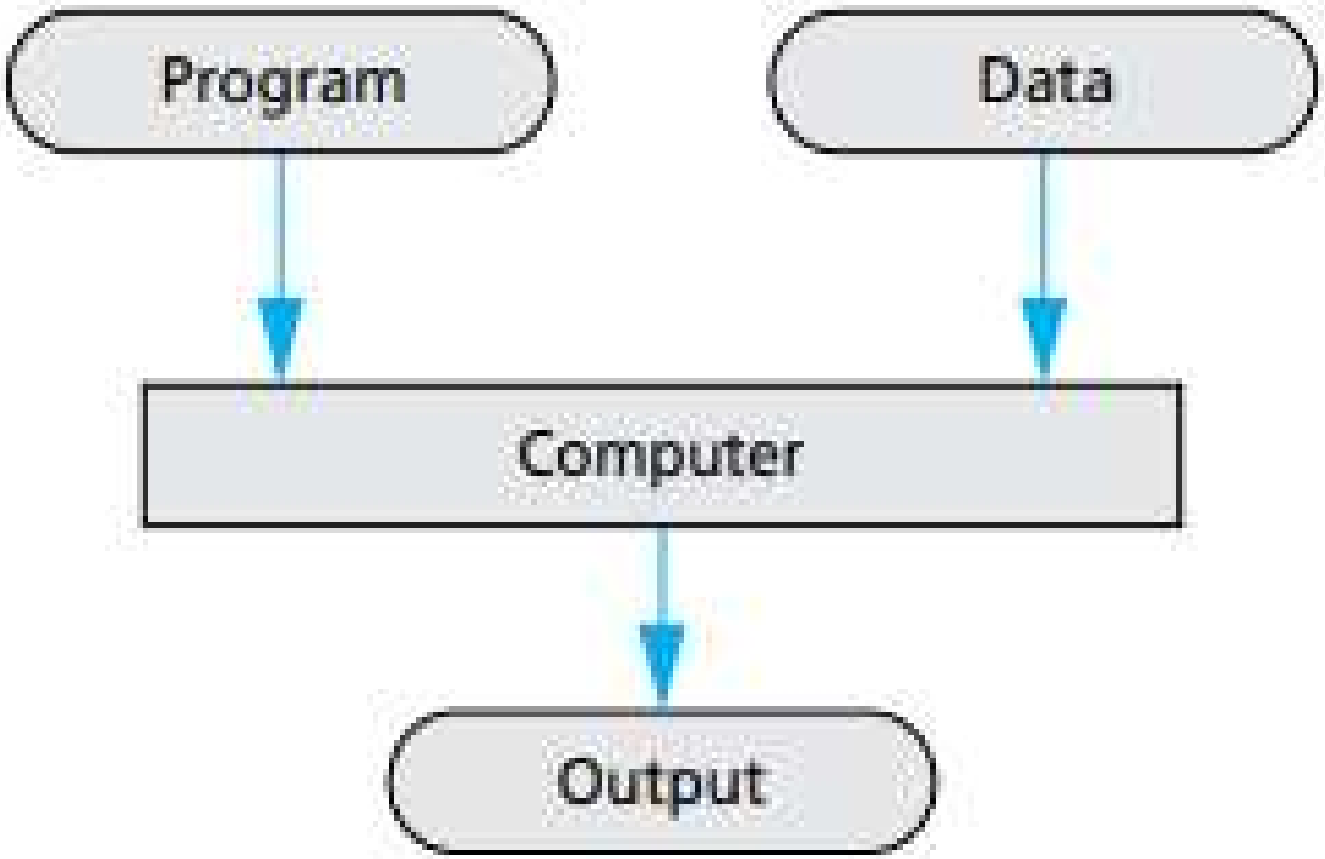
GADAA DIGITAL YouTube

Computer Programming

- A computer program is a set of instructions written in a programming language that tells a computer what tasks to perform.
- These instructions can range from simple calculations to complex operations, and they are executed by the computer's processor according to the program's logic.
- Programs can perform a wide variety of functions, such as managing data, controlling hardware devices, running applications, or executing algorithms.
- The concepts of computer programming are logical and mathematical in nature.
- The general purpose of a program is:
  - ✓ Receive input
  - ✓ Perform some calculation/manipulation of the data
  - ✓ Produce output

GADAA DIGITAL YouTube

Computer Programming



GADAA DIGITAL
YouTube

Computer Programming

- A computer program is a set of instructions written in a programming language that tells a computer what tasks to perform.
- These instructions can range from simple calculations to complex operations, and they are executed by the computer's processor according to the program's logic.
- Programs can perform a wide variety of functions, such as managing data, controlling hardware devices, running applications, or executing algorithms.
- The concepts of computer programming are logical and mathematical in nature.
- The general purpose of a program is:
  - ✓ Receive input
  - ✓ Perform some calculation/manipulation of the data
  - ✓ Produce output

GADAA DIGITAL YouTube

Computer Programming

	Human Language	Programming Language
Uses	for sharing ideas, emotions, stories, questions, and social bonding	to give a sequence of unambiguous instructions to a computer to perform a task
Goal	The goal can be persuasive, informative, artistic, or relational	The goal is to achieve a specific, functional outcome, such as calculating a value, storing data, or displaying a webpage
Audience	Other humans	Primarily machines, secondarily humans
Syntax and Grammar (The Rules)	Complex and Tolerant of Errors	Precise and Intolerant of Errors: Syntax is rigid

GADAA DIGITAL YouTube

Computer Programming

- Programming languages used by programmer to communicate with computers.
- It is set of instructions written in any specific languages (C, C++, java, python,…) to perform specific task.
- Programming language is mainly used to develop desktop applications, websites, and mobile applications.
- Programming languages divided into three major categories
  - ✓ Low-level programming language
  - ✓ Middle-level programming language
  - ✓ High-level programming language

GADAA DIGITAL YouTube

Computer Programming

1. **Low level programming language**

- Are closer to the binary codes(0s and 1s) that computer's processor understands
- They provide little to no abstraction from hardware
- Programmer must manage memory and processor registers manually
- Fast and efficient because code is written for specific machine
- Difficult to read, write, and debug for humans
- Directly manipulates hardware component(registers, memory address)
- Code is not portable

GADAA DIGITAL YouTube

Computer Programming

1. **Low level programming language**

- There are two main types of low level programming languages

A. Machine code

- Absolutely lowest level
- It contain pure binary instructions(0s and 1s) that are directly executed by the CPU.
- No one write in raw machine code today

GADAA DIGITAL YouTube

Computer Programming

1. **Low level programming language**

- There are two main types of low level programming languages

B. Assembly language

- Uses short, mnemonic codes like ADD, MOV,SUB to represent machine code instructions.
- An assembler translates this code into machine code
- Eg: simple program to add two numbers like below

```
MOV AL, 5     ; move value 5 to AL register
MOV BL, 3     ; move value 3 5o BL register
ADD AL, BL    ; add the value in BL to value in AL
```

Real-world use
- Device driver
- Firmware for embedded system

GADAA DIGITAL YouTube

Computer Programming

2. **Middle level programming language**

- This is somewhat informal category
- Languages in this tier provide many of the comforts of high-level languages like structured programming, loops, and functions
- But still allow for a significant degree of direct hardware manipulation similar to low-level languages
- More readable and structured than low level languages
- Allow user for manual memory management
- Used for both system programming and application programming
- Eg: C

GADAA DIGITAL YouTube

Computer Programming

3. **High level programming language**

- Are designed to be easy for human to read, write, and understand
- They feature strong abstraction, meaning the programmer doesn't need to know anything about underlying hardware
- They use nature language elements and manage resource automatically
- Easy to learn, read and write
- Automatic memory management
- Portable
- Use interpreter and compiler to translate to target code
- Eg: Python, Java, JavaScript, C#, Ruby, PHP, Swift, Kotlin etc

GADAA DIGITAL YouTube

Computer Programming

Summary on types of programming languages

Feature	Low level(Assembly)	Middle Level(C)	High Level(Python)
Abstraction	Minimal	Moderate	High
Ease of use	Very difficult	Moderate	Very easy
Speed	Fastest	Very fast	Slower
Memory mgt	Manual	Manual	Automatic
Portability	None	Good(with compiler)	Excellent
Common use	Drivers, Firmware	OS kernels, Games	Web App, Data science

GADAA DIGITAL YouTube

Development Tools

- **Editor**: allows the programmer to enter the program source code and save it to files.
- **Compilers**: translates the source code to target code. Compilers translate the contents of a source file and produce a file containing all the target code. Popular compiled languages include C, C++, Java, C#.
- **Interpreters**: is like a compiler, in that it translates higher-level source code into target code. interpreter translates source code statements into machine language each time a user runs the program.
  - ✓ compiled programs execute more quickly than interpreted programs because the translation activity occurs only once.
- **Debugger**: allows a programmer to more easily trace a program's execution in order to locate and correct errors in the program's implementation.
- **IDEs**: An IDE includes editors, debuggers, and other programming aids in one comprehensive program.
- Eg. DEV++, Falcon, VS Code, Visual Studio, Netbean,…

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

1. How compiler work

- Compiler reads the entire source code program and translates it into an equivalent program in target code before execution
- The output is executable language that can be run independently
- The compilation process is typically broken into several phases:

i. Lexical Analysis (Scanning)

- The source code is read as a stream of characters.
- It is grouped into meaningful sequences called tokens (keywords, identifiers, operators, literals).
- Example: int a = 5 + b; → tokens: int, a, =, 5, +, b, ;

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

ii. Syntax Analysis (Parsing)

- Tokens are arranged into a parse tree (or syntax tree) that reflects the grammatical structure of the language.
- This phase checks for syntax errors (e.g., missing semicolons, mismatched parentheses).

iii. Semantic Analysis

- Checks for logical consistency (type checking, variable declaration before use).
- Annotates the parse tree with type information (creating an annotated syntax tree).

iv. Intermediate Code Generation

- The compiler produces an intermediate representation (IR) that is easier to optimize and translate further.
- IR is independent of the target machine (e.g., three-address code).

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

v. Optimization

- The intermediate code is improved to make it faster, smaller, or more efficient.
- Examples: eliminating dead code, constant folding, loop optimizations.

vi. Code Generation

- The optimized IR is translated into the target machine language (assembly or binary).
- This phase assigns registers, selects instructions, and produces relocatable machine code.

vii. Linking (often separate)

- If the program uses external libraries, the linker combines the compiled object code with those libraries into a single executable.

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

2. How an Interpreter works

- An interpreter directly executes the instructions in the source code without producing a separate executable file.
- It reads, analyses, and executes the code line by line (or statement by statement).

❖ Basic Interpreter Loop

- ✓ Read one statement of source code.
- ✓ Parse it (similar lexical/syntax analysis as a compiler).
- ✓ Perform the actions specified (e.g., evaluate expressions, call functions).
- ✓ Repeat until the program ends.

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

- Key Differences Summarized

Feature	Compiler	Interpreter
Translation	Entire program at once, before execution	Line by line, during execution
Output	Produces standalone executable (machine code).	No standalone output; executes directly.
Speed	Faster execution (code already translated)	Slower execution (repeated translation).
Error detection	All errors reported after full compilation	Stops at first error in a line.
Memory usage	Generally more (due to intermediate files).	Less (no separate executable)
Portability	Machine-dependent (must recompile for each)	Portable (interpreter per platform).

GADAA DIGITAL YouTube

How Compiler and Interpreter work?

- Key Differences Summarized

Compiler	Interpreter
C++ Source Code	Python Script

GADAA DIGITAL
YouTube

GADAA DIGITAL

CHAPTER TWO

Digital System

Boolean Algebra and Logic Gates